

모델 최적화

1. 과적합(Overfitting)과 과소적합(Underfitting)

✓ 과적합(Overfitting)

- **정의:** 모델이 학습 데이터에 지나치게 맞춰져서 새로운 데이터(테스트 데이터)에서 성능이 저하되는 현상
- **특징:**
 - 학습 데이터에서는 높은 정확도를 보이지만, 테스트 데이터에서는 성능이 떨어짐
 - 모델이 노이즈까지 학습하여 일반화 능력이 부족함

모델 최적화

- 해결 방법:

- **데이터 증강(Data Augmentation):** 학습 데이터를 다양하게 변형하여 데이터의 다양성을 높임
- **정규화(Regularization):** L1, L2 정규화 기법을 사용하여 모델이 가중치를 과도하게 조정하지 않도록 함
- **드롭아웃(Dropout):** 신경망의 일부 뉴런을 랜덤하게 제거하여 모델의 복잡도를 줄임
- **조기 종료(Early Stopping):** 검증 데이터의 성능이 더 이상 향상되지 않으면 학습을 조기에 중단
- **모델 단순화:** 지나치게 복잡한 모델(너무 많은 층이나 뉴런)을 줄이고 적절한 크기의 모델을 선택

✓ 과소적합(Underfitting)

- **정의:** 모델이 학습 데이터의 패턴을 충분히 학습하지 못하여 성능이 낮은 상태
- **특징:**
 - 학습 데이터와 테스트 데이터 모두에서 성능이 낮음
 - 데이터의 복잡성을 충분히 반영하지 못함

- 해결 방법:

- **더 복잡한 모델 사용:** 신경망의 층을 추가하거나 뉴런 개수를 증가시켜 표현력을 높임
- **더 많은 특징 사용(Feature Engineering):** 데이터에 포함된 유용한 특성을 더 잘 반영할 수 있도록 특징을 추가
- **더 많은 학습 데이터 확보:** 모델이 충분히 학습할 수 있도록 데이터셋을 늘리거나 고품질 데이터를 확보
- **더 오래 학습:** 학습 에포크(epoch)를 증가하여 모델이 충분히 학습할 시간을 줌

2. 하이퍼파라미터 튜닝(Hyperparameter Tuning)

✓ 하이퍼파라미터란?

- 모델이 학습하는 동안 직접 조정되지 않고 사람이 설정해야 하는 값
- 대표적인 하이퍼파라미터:
 - 학습률(Learning Rate)
 - 배치 크기(Batch Size)
 - 은닉층 개수와 뉴런 수
 - 드롭아웃 비율(Dropout Rate)
 - L1, L2 정규화 값
 - 최적화 알고리즘(Adam, SGD 등)

✅ L1 정규화(Lasso Regularization)와 L2 정규화(Ridge Regularization)

정규화(Regularization)는 모델이 과적합(Overfitting)되는 것을 방지하고 일반화 성능을 향상시키기 위해 사용되는 기법이다. 그중 **L1 정규화(Lasso)**와 **L2 정규화(Ridge)**는 가장 많이 사용되는 두 가지 방법이다.

1 L1 정규화 (Lasso Regularization)

🏷 개념

- L1 정규화는 손실 함수(Loss Function)에 가중치들의 절댓값의 합을 추가하는 방식이다.
- 불필요한 가중치(Weight)를 0으로 만들어 **희소 모델(Sparse Model)**을 생성하는 특징이 있다.

수식

L1 정규화가 적용된 손실 함수:

$$L = \text{Loss} + \lambda \sum |w|$$

- **Loss**: 원래의 손실 함수(예: MSE, Cross-Entropy 등)
- **λ (람다, Regularization Strength)**: 정규화 강도를 조절하는 하이퍼파라미터 (값이 클수록 가중치 감소 효과가 커짐)
- **w**: 모델의 가중치들

특징

- ✓ 불필요한 가중치(Weight)를 0으로 만들기 때문에 변수 선택(Feature Selection) 효과가 있음
- ✓ 희소 모델을 생성하여 차원이 높은 데이터에서 유용함
- ✓ 일부 특징이 완전히 제거될 수 있음 → 가중치가 0이 됨

L1 정규화 적용 예제 (Python - Scikit-Learn)

python

```
from sklearn.linear_model import Lasso
```

```
# Lasso 회귀 모델 생성 ( $\lambda=0.1$ )
```

```
lasso = Lasso(alpha=0.1)
```

```
lasso.fit(X_train, y_train)
```

```
# 가중치 확인
```

```
print(lasso.coef_) # 일부 가중치가 0이 될 가능성이 높음
```

2 L2 정규화 (Ridge Regularization)

개념

- L2 정규화는 손실 함수에 가중치들의 제곱합을 추가하는 방식이다.
- 가중치를 0에 가깝게 만들지만 완전히 0으로 만들지는 않음 → 가중치를 부드럽게 줄여줌 (Shrinkage 효과)

수식

L2 정규화가 적용된 손실 함수:

$$L = \text{Loss} + \lambda \sum w^2$$

- **Loss**: 원래의 손실 함수(예: MSE, Cross-Entropy 등)
- **λ (람다, Regularization Strength)**: 정규화 강도 조절
- **w** : 모델의 가중치들

모델 최적화

✓ 특징

- ✓ 모든 가중치를 0에 가깝게 줄이지만 완전히 0이 되지는 않음
- ✓ 다중공선성(변수 간 상관관계가 높을 때 발생하는 문제)을 완화하는 효과가 있음
- ✓ 모든 변수(Feature)를 활용해야 하는 경우 적합

🎯 L2 정규화 적용 예제 (Python - Scikit-Learn)

python

```
from sklearn.linear_model import Ridge

# Ridge 회귀 모델 생성 ( $\lambda=0.1$ )
ridge = Ridge(alpha=0.1)
ridge.fit(X_train, y_train)

# 가중치 확인
print(ridge.coef_) # 모든 가중치가 작아지지만 0이 되지는 않음
```

3 L1 정규화 vs L2 정규화 비교

구분	L1 정규화 (Lasso)	L2 정규화 (Ridge)
가중치 효과	일부 가중치를 완전히 0으로 만듦 (Feature Selection)	모든 가중치를 작게 만들지만 0이 되지는 않음
희소성 (Sparsity)	희소성 O (일부 특성 제거)	희소성 X (모든 특성 유지)
사용 사례	중요 특성만 남기고 싶을 때 (차원 축소)	모든 변수를 고려하는 모델이 필요할 때
다중공선성 해결	부분적으로 해결	효과적으로 해결

4 L1 + L2 정규화 (ElasticNet)

- Lasso의 변수 선택 효과 + Ridge의 가중치 감소 효과를 결합한 방법
- 손실 함수:

$$L = \text{Loss} + \lambda_1 \sum |w| + \lambda_2 \sum w^2$$

ElasticNet 적용 예제 (Python - Scikit-Learn)

python

 복사

```
from sklearn.linear_model import ElasticNet

# ElasticNet 모델 생성 (L1과 L2 혼합)
elasticnet = ElasticNet(alpha=0.1, l1_ratio=0.5) # l1_ratio: L1과 L2 비율 조절
elasticnet.fit(X_train, y_train)

# 가중치 확인
print(elasticnet.coef_)
```

결론

- **L1 정규화:** 희소 모델을 원할 때, 중요하지 않은 특징을 제거하고 싶을 때 사용
- **L2 정규화:** 모든 특징을 고려하면서 가중치를 부드럽게 줄이고 싶을 때 사용
- **ElasticNet:** 두 개의 장점을 결합하여 변수 선택과 가중치 감소 효과를 동시에 활용 가능

✓ 하이퍼파라미터 튜닝 방법

1) 그리드 탐색(Grid Search)

- 여러 하이퍼파라미터 조합을 정해두고 전부 실험하여 최적의 조합을 찾는 방식
- **장점:** 가장 최적의 조합을 찾을 확률이 높음
- **단점:** 계산 비용이 큼 (모든 조합을 시도해야 하기 때문)

2) 랜덤 탐색(Random Search)

- 일정 범위에서 하이퍼파라미터 값을 무작위로 선택하여 실험하는 방식
- **장점:** 계산 비용이 그리드 탐색보다 낮고, 예상보다 좋은 조합을 찾을 수도 있음
- **단점:** 최적의 조합을 놓칠 수도 있음

3) 베이지안 최적화(Bayesian Optimization)

- 이전 실험 결과를 기반으로 가장 가능성이 높은 영역을 탐색하는 방식
- **장점:** 효율적이고 계산 비용이 상대적으로 적음
- **단점:** 구현이 복잡할 수 있음

4) 하이퍼밴드(Hyperband)

- 여러 모델을 동시에 학습시키면서 성능이 낮은 모델을 빠르게 제거하여 시간을 절약하는 기법
- **장점:** 빠르게 최적의 모델을 찾을 수 있음
- **단점:** 일부 경우에는 최적의 결과를 찾기 어려울 수도 있음

3. 모델 최적화 실습

✓ 실습 목표

- 하이퍼파라미터 튜닝을 통해 과적합과 과소적합을 방지하고 최적의 모델을 만드는 과정 실습

✓ 실습 과정 (예제: TensorFlow/Keras 사용)

1 데이터 준비

python

📄 복사 ✎ 편집

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.datasets import mnist

# 데이터 불러오기
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0 # 정규화
```

2 기본 모델 구성

python

```
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

모델 최적화

3 하이퍼파라미터 튜닝 예제 (Keras Tuner 활용)

python

📄 복사 ✎ 편집

```
import keras_tuner as kt

def build_model(hp):
    model = keras.Sequential([
        keras.layers.Flatten(input_shape=(28, 28)),
        keras.layers.Dense(hp.Int('units', min_value=32, max_value=256, step=32), activation='relu'),
        keras.layers.Dropout(hp.Float('dropout', min_value=0.1, max_value=0.5, step=0.1)),
        keras.layers.Dense(10, activation='softmax')
    ])

    model.compile(
        optimizer=keras.optimizers.Adam(hp.Choice('learning_rate', [0.001, 0.0001, 0.0005])),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy'])

    return model
```

모델 최적화

```
tuner = kt.RandomSearch(  
    build_model,  
    objective='val_accuracy',  
    max_trials=5,  
    executions_per_trial=1,  
    directory='my_dir',  
    project_name='intro_to_kt'  
)  
  
tuner.search(x_train, y_train, epochs=10, validation_split=0.2)
```

모델 최적화

4 최적의 하이퍼파라미터 찾기

python

📄 복사 ✎ 편집

```
best_hps = tuner.get_best_hyperparameters(num_trials=1)[0]  
print(f"Best units: {best_hps.get('units')}, Best dropout: {best_hps.get('dropout')}, Best learning_rate: {best_hps.get('learning_rate')}")
```

5 최적의 모델 학습

python

📄 복사 ✎ 편집

```
best_model = tuner.hypermodel.build(best_hps)  
best_model.fit(x_train, y_train, epochs=20, validation_split=0.2)
```

결론

- **과적합과 과소적합을 방지하는 방법을 이해하고 이를 해결하는 전략(정규화, 드롭아웃, 조기 종료 등)을 학습함**
- **하이퍼파라미터 튜닝을 통해 최적의 모델을 찾는 방법을 배움 (그리드 탐색, 랜덤 탐색, 베이지안 최적화 등)**
- **실습을 통해 모델 성능을 최적화하는 방법을 체험하고, Keras Tuner를 사용하여 최적의 모델을 자동으로 찾는 과정 이해**

Scikit-learn을 사용하여 머신 러닝 모델 파인 튜닝(fine-tuning) 실습

[예제 개요]

데이터셋: digits (손글씨 숫자 이미지 데이터)

모델: 랜덤 포레스트 (RandomForestClassifier)

파인 튜닝 방법: GridSearchCV를 사용하여 하이퍼파라미터 최적화

최적화할 하이퍼파라미터: n_estimators, max_depth, min_samples_split

평가 지표: 정확도(Accuracy)

🔥 설명

1. 데이터 로드 및 분할

- `digits` 데이터셋을 가져와서 훈련/테스트 데이터로 분할

2. 기본 랜덤 포레스트 모델 학습

- `RandomForestClassifier` 를 사용하여 기본 모델 학습 후 정확도 측정

3. `GridSearchCV`를 통한 하이퍼파라미터 최적화

- `n_estimators`, `max_depth`, `min_samples_split` 을 조합하여 최적의 모델 탐색

4. 최적 모델로 성능 비교

- 최적화된 모델과 기존 모델의 성능 비교

5. 데이터 시각화

- 일부 숫자 이미지를 출력하여 데이터 확인

파인 튜닝(fine-tuning) 실습 질의 :

아래 내용으로 Scikit-learn을 사용하여 머신 러닝 모델을 파인 튜닝(fine-tuning) 하는 모델을 만들어줘.

데이터셋: digits (손글씨 숫자 이미지 데이터)

모델: 랜덤 포레스트 (RandomForestClassifier)

파인 튜닝 방법: GridSearchCV를 사용하여 하이퍼파라미터 최적화

최적화할 하이퍼파라미터: n_estimators, max_depth, min_samples_split

평가 지표: 정확도(Accuracy)

결과 출력

```
</> Gridsearch Rf Digits
15
16 # 하이퍼파라미터 그리드 설정
17 param_grid = {
18     'n_estimators': [50, 100, 200],
19     'max_depth': [None, 10, 20, 30],
20     'min_samples_split': [2, 5, 10]
21 }
22
23 # GridSearchCV 수행
24 grid_search = GridSearchCV(rf, param_grid, cv=5, scoring='accuracy', n_jobs=-1)
25 grid_search.fit(X_train, y_train)
26
27 # 최적의 하이퍼파라미터 출력
28 print("Best Parameters:", grid_search.best_params_)
29
30 # 최적 모델로 테스트 데이터 예측
```

대신 채팅에서 답합니다 >

이 코드는 `GridSearchCV` 를 사용하여 `RandomForestClassifier` 의 최적 하이퍼파라미터를 찾고, 최적의 모델로 테스트 데이터를 평가하여 정확도를 출력합니다. 필요한 경우 하이퍼파라미터 범위를 조정하여 성능을 더 최적화할 수도 있습니다. 추가로 시각화나 피쳐 중요도 분석이 필요하다면 알려주세요! 🚀

감사합니다